```
    public Task HandleAsync(AuthorizationHandlerContext context)
    {
        foreach (var requirement in                                    ❶
        ➥ context.PendingRequirements.ToList())                       ❶
        {
            if (requirement is ViewRolesRequirement req)               ❷
            {
                var joiningDateClaim =
                ➥ context.User.FindFirst(c => c.Type ==
                ➥ "Joining Date")?.Value;
                if (joiningDateClaim == null)
                {
                    return Task.CompletedTask;
                }
                var joiningDate = Convert.ToDateTime(joiningDateClaim);

                if (context.User.HasClaim("Permission", "View Roles") &&
                        joiningDate < DateTime.Now.AddMonths(req.Months))
                {
                    context.Succeed(requirement);
                }
            }
        }
        return Task.CompletedTask;
    }
}
```

❶ The PendingRequirements return all unsatisfied requirements. They need to be assigned to a list, so you can execute operations against them.

❷ Use pattern matching to identify ViewRolesRequirements and assign them to a local variable.

The final difference between this approach and the combined requirement-handler is that you need to register the handler with the service container as an implementation of `IAuthorizationHandler`:

```
builder.Services.AddSingleton<IAuthorizationHandler, ViewRolesHandler>();
```

Once you have done this, the authorization policy works in the same way as the previous approach that employed the combined requirement-handler combo.